HDOC Day-62 | C Programming

Topic: Merge Two Sorted Arrays Using Pointers

Problem: Read the sizes and elements of two arrays already sorted in ascending order. Pass their addresses to **merge_sorted_arrays()**, merge them into a third array using two pointers, preserve duplicates, and display the merged values separated by spaces.

C Program

```
#include <stdio.h>

void merge_sorted_arrays(int *arr1, int n1,
                         int *arr2, int n2) {
    int merged[n1 + n2];
    int i = 0, j = 0, k = 0;

    while (i < n1 && j < n2) {
        if (*(arr1 + i) <= *(arr2 + j))
            merged[k++] = *(arr1 + i++);
        else
            merged[k++] = *(arr2 + j++);
    }

    while (i < n1)
        merged[k++] = *(arr1 + i++);
    while (j < n2)
        merged[k++] = *(arr2 + j++);

    printf("Merged array: ");
    for (i = 0; i < k; i++)
        printf("%d ", merged[i]);
    printf("\n");
}

int main(void) {
    int n1, n2, i;
    printf("Enter size of first array: ");
    scanf("%d", &n1);
    printf("Enter size of second array: ");
    scanf("%d", &n2);

    if (n1 <= 0 || n2 <= 0) {
        printf("Sizes must be positive.\n");
        return 1;
    }

    int arr1[n1], arr2[n2];
    printf("Enter sorted first array: ");
    for (i = 0; i < n1; i++)
        scanf("%d", &arr1[i]);

    printf("Enter sorted second array: ");
    for (i = 0; i < n2; i++)
        scanf("%d", &arr2[i]);

    merge_sorted_arrays(arr1, n1, arr2, n2);
    return 0;
}
```

Sample Input and Output

```
Enter size of first array: 4
Enter size of second array: 5
Enter sorted first array: 1 3 5 7
Enter sorted second array: 2 3 6 8 9
Merged array: 1 2 3 3 5 6 7 8 9
```

Explanation: Indices i and j track the current elements of the two sorted arrays. The smaller element is copied into merged[]. After one array finishes, the remaining elements of the other array are copied. Equal values are retained.

Time Complexity: O(n1 + n2) **Extra Space:** O(n1 + n2)